🧮 Mathematical Foundations of the System

This document outlines the core dynamic equations and control theory implemented in this repository.

1. Kinematic State Estimation (1D Kalman Filter)

Industrial robotic arms typically provide high-resolution joint position (encoder) feedback q, but lack direct velocity sensors. Using simple first-order differentiation ($\dot{q} \approx \frac{\Delta q}{\Delta t}$) at 1000Hz can amplify noise, rendering torque control unstable.

We implement a discrete 1D Linear Kalman Filter for each joint to optimally fuse the noisy position measurements with the kinematic acceleration commands (used as the control input).

State Vector:

$$x_k = \begin{bmatrix} q_k \\ \dot{q}_k \end{bmatrix}$$

Prediction Step (Kinematic Model):

$$x_{k|k-1} = F x_{k-1|k-1} + B u_k$$

$$P_{k|k-1} = F P_{k-1|k-1} F^T + Q$$

Where the state transition matrix F and control matrix B are defined by standard kinematics (with $dt = 0.001s$ and input $u_k = \ddot{q}_{target}$):

$$F = \begin{bmatrix} 1 & dt \\ 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} \frac{1}{2}dt^2 \\ dt \end{bmatrix}$$

Q is the process noise covariance matrix representing uncertainty in the acceleration execution.

Update Step (Measurement):

$$K_k = P_{k|k-1} H^T (H P_{k|k-1} H^T + R)^{-1}$$

$$x_{k|k} = x_{k|k-1} + K_k(z_k - H x_{k|k-1})$$

$$P_{k|k} = (I - K_k H) P_{k|k-1}$$

Where the observation matrix is $H = \begin{bmatrix} 1 & 0 \end{bmatrix}$, z_k is the raw encoder reading, and R is the measurement noise variance.

2. Rigid Body Dynamics and Friction Modeling

The standard equation of motion for an n-DOF rigid body manipulator is calculated using the **Recursive Newton-Euler Algorithm (RNEA)** via the Pinocchio library:

$$\tau_{rnea} = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q)$$

Where:

- $M(q) \in \mathbb{R}^{n \times n}$ is the symmetric, positive-definite mass/inertia matrix.
- $C(q, \dot{q})\dot{q} \in \mathbb{R}^n$ represents the Coriolis and centrifugal forces.
- $G(q) \in \mathbb{R}^n$ is the gravity vector.

Motor Dynamics and Friction:

To achieve realistic simulation, we must account for the motor's rotor inertia (amplified by the harmonic drive gear ratio squared) and joint friction:

$$\tau_{motor} = I_a \ddot{q}$$

$$\tau_{fric} = F_v \dot{q} + F_c \text{sgn}(\dot{q})$$

Where I_a is the armature inertia, F_v is viscous friction, and F_c is Coulomb (dry) friction.

Friction Compensation: Anti-Chattering & Stiction Breakaway

The standard theoretical model for Coulomb (dry) friction is $F_c \text{sgn}(\dot{q})$. However, implementing this directly in a discrete 1000Hz control loop introduces two problems: **Chattering** and **Steady-State Stall (Stiction)**. Our friction compensator overcomes both using a novel error-injected hyperbolic tangent function:

$$\tau_{fric} = F_v \dot{q} + F_c \tanh\left(c \cdot (\dot{q}_{target} + \lambda e)\right)$$

1. Overcoming Chattering via Smooth Approximation ($\tanh$)

The standard signum function $\text{sgn}(\dot{q})$ is discontinuous at $\dot{q} = 0$. In real-world systems, sensor noise ensures that the velocity $\dot{q}$ is rarely exactly zero; it fluctuates rapidly between positive and negative infinitesimal values. This causes the standard Coulomb compensator to violently alternate between $+F_c$ and $-F_c$ at high frequencies.

To solve this, we replace the discontinuous sgn function with a smooth hyperbolic tangent function ($\tanh$). The parameter $c = 300.0$ dictates the slope of the transition region. It acts as a stiff spring near zero velocity, providing a smooth continuous torque transition across the origin while rapidly saturating to ± 1 at higher velocities, mirroring ideal Coulomb behavior without the destructive high-frequency switching.

2. Overcoming Steady-State Stall via Error Injection (λe)

While the $\tanh$ modification solves chattering, it introduces a new problem at zero velocity. If the robot reaches a standstill ($\dot{q} \approx 0$) but is slightly away from the target position ($e \neq 0$), the PD control law outputs a small corrective torque ($K_p e$).

However, because the velocity is near zero, the smoothed friction compensator outputs almost zero ($\tanh(0) = 0$). Consequently, the small PD torque is entirely absorbed by the physical static friction (stiction) of the gearbox. The robot becomes "stuck" just millimeters away from its target, resulting in a persistent **steady-state error**.

To eliminate this, we base the Coulomb friction calculation not on the noisy actual velocity, but on a **fictitious reference velocity**: $(\dot{q}_{target} + \lambda e)$.

- **Feedforward:** When moving, the expected target velocity $\dot{q}_{target}$ drives the friction compensation seamlessly.

- **Stiction Breakaway (The λe term):** When the robot is supposed to be stationary ($\dot{q}_{target} = 0$) but a position error exists ($e \neq 0$), the term λe acts as a fictitious velocity. The scaling factor $\lambda = 5.0$ amplifies this positional error, pushing the $\tanh$ function into its saturation region.
- **The Result:** The controller actively outputs a directional friction-cancellation torque ($\pm F_c$) *in the direction of the error*. This "boost" precisely cancels the mechanical breakaway friction, allowing the small PD torque to pull the robot effortlessly to the exact target. Once the error reaches zero ($e = 0$), the fictitious velocity vanishes, the $\tanh$ evaluates to zero, and the robot rests perfectly stationary.

3. Computed Torque Control (CTC) Law

The goal of the CTC is to dynamically cancel out the nonlinearities of the robot and enforce a linear, decoupled, second-order error response.

Define the joint tracking error and its derivative:

$$e = q_{target} - q$$

$$\dot{e} = \dot{q}_{target} - \dot{q}$$

We define the "resolved acceleration" a_d using a Proportional-Derivative (PD) feedback loop combined with feedforward acceleration:

$$a_d = \ddot{q}_{target} + K_p e + K_v \dot{e}$$

The final control torque sent to the joint motors is:

$$\tau_{cmd} = M(q)a_d + C(q, \dot{q})\dot{q} + G(q) + I_a a_d + \tau_{fric}$$

Error Dynamics Proof:

If our dynamic model is highly accurate ($\hat{M} = M$, etc.), substituting the control law into the robot's equation of motion yields:

$$(M(q) + I_a)(\ddot{q}_{target} - \ddot{q} + K_p e + K_v \dot{e}) = 0$$

Since $(M(q) + I_a)$ is positive-definite and invertible, we obtain the closed-loop error dynamics:

$$\ddot{e} + K_v \dot{e} + K_p e = 0$$

By choosing strictly positive diagonal gain matrices K_p and K_v, the error $e(t)$ converges exponentially to zero.

4. System Identification (Fourier Series & Least Squares)

Parameters such as the mass, inertia, and center of mass of the robot body are known at the time of manufacture; however, parameters like friction and rotor inertia are difficult to measure precisely, necessitating system identification to estimate them. To find the parameters I_a, F_v, F_c, we execute a continuous excitation trajectory modeled by a finite Fourier series ($N_f = 7$, base frequency $w = 2\pi f_0$):

$$q_i(t) = \sum_{l=1}^{N_f} \left[\frac{a_{i,l}}{wl} \sin(wlt) - \frac{b_{i,l}}{wl} \cos(wlt) + \frac{b_{i,l}}{wl} \right]$$

We impose boundary conditions such that $q(0), \dot{q}(0), \ddot{q}(0) = 0$ to ensure smooth start and stop transitions.

Linear Regressor Formulation:

The dynamics equation can be rewritten linearly with respect to the unknown parameter vector θ:

$$\tau_{cmd} - \tau_{rnea}(q, \dot{q}, \ddot{q}) = I_a \ddot{q} + F_v \dot{q} + F_c \tanh(c\dot{q})$$

Let $y = \tau_{cmd} - \tau_{rnea}$ and the regressor matrix $W = \begin{bmatrix} \ddot{q} & \dot{q} & \tanh(c\dot{q}) \end{bmatrix}$. The unknown parameters are $\theta = \begin{bmatrix} I_a & F_v & F_c \end{bmatrix}^T$.

For K recorded samples during the excitation trajectory:

$$Y = W\theta$$

We solve for θ using Constrained Ordinary Least Squares (to ensure physical parameters remain strictly positive $\theta \geq 0$):

$$\theta^* = \arg\min_{\theta \geq 0} ||W\theta - Y||_2^2$$

5. Kinematic E-Stop (Deceleration Trajectory)

When a software E-Stop is triggered, simply zeroing the torque causes the robot to collapse under gravity, while instantly zeroing the velocity causes infinite deceleration (jerk), which can destroy the harmonic drive gearboxes.

Instead, the controller intercepts the E-Stop signal and generates a real-time kinematic deceleration profile bounded by a maximum safe deceleration limit a_{max} (e.g., $5\times$ the nominal trajectory acceleration limits).

For a joint currently moving at velocity $\dot{q}(k)$, the deceleration command is set in direct opposition to the movement:

$$\ddot{q}_{cmd} = -\text{sgn}(\dot{q}(k)) a_{max}$$

Under normal deceleration, the discrete position and velocity targets are updated iteratively during the control loop ($dt = 0.001s$):

$$q_{target}(k+1) = q_{target}(k) + \dot{q}(k)dt + \frac{1}{2}\ddot{q}_{cmd}dt^2$$

$$\dot{q}_{target}(k+1) = \dot{q}(k) + \ddot{q}_{cmd}dt$$

Zero-Crossing Prevention (Anti-Jitter):

A critical edge case in discrete-time integration occurs during the final time step before the robot completely stops. If the applied constant deceleration is large enough, integrating over the full time step dt will cause the velocity to overshoot zero and flip its sign. In the next time step, the controller would apply deceleration in the opposite direction, leading to high-frequency jitter (chattering) around the zero-velocity point.

To prevent this, the controller calculates a tentative next velocity:

$$\dot{q}_{next} = \dot{q}(k) + \ddot{q}_{cmd} dt$$

If a zero-crossing is detected (i.e., the sign of $\dot{q}_{next}$ is different from $\dot{q}(k)$, or it reaches exactly zero), the controller abandons the standard dt integration. Instead, it calculates the exact fractional time t_{zero} required for the velocity to reach absolute zero:

$$t_{zero} = \left| \frac{\dot{q}(k)}{\ddot{q}_{cmd}} \right|$$

Since $t_{zero} \leq dt$, the integration step is truncated exactly at this fractional time, ensuring the joint smoothly locks at a complete standstill without overshooting:

$$q_{target}(k+1) = q_{target}(k) + \dot{q}(k) t_{zero} + \frac{1}{2} \ddot{q}_{cmd} t_{zero}^2$$

$$\dot{q}_{target}(k+1) = 0$$

$$\ddot{q}_{cmd} = 0$$

This logic guarantees that the manipulator strictly adheres to maximum mechanical stress tolerances during emergency interventions, while resting completely stationary once the kinetic energy has been safely dissipated.